“Linux Inter-Process Communication Using Pipes and FIFOs”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

B. Deepak	**2520080035**
E. Pranay Reddy	**2520080061**
A. Lakshmi Sai Teja	**2520080005**

Under the Esteemed Guidance of

Mr. Raghupathi Manthena
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux Inter-Process Communication Using Pipes and FIFOs"** is a record of Bonafide work of **B. Deepak - 2520080035, E. Pranay Reddy - 2520080061, A. Lakshmi Sai Teja - 2520080005** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B. Deepak	**2520080035**
E. Pranay Reddy	**2520080061**
A. Lakshmi Sai Teja	**2520080005**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **"Linux Inter-Process Communication Using Pipes and FIFOs"** is a bonafide work done and submitted by **B. Deepak - 2520080035, E. Pranay Reddy - 2520080061, A. Lakshmi Sai Teja - 2520080005**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Mr. Raghupathi Manthena**, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

B. Deepak 2520080035

E. Pranay Reddy 2520080061

A. Lakshmi Sai Teja 2520080005

Abstract

Linux Inter-Process Communication Using Pipes and FIFOs is a project designed to demonstrate how different processes communicate and exchange information within a Linux operating system. Inter-Process Communication (IPC) is an important concept in operating systems that enables processes to share data, coordinate their activities, and work together efficiently. This project focuses mainly on two IPC mechanisms: Pipes and FIFOs (Named Pipes). Pipes provide a simple communication channel between related processes, such as a parent process and its child process. One process writes data into the pipe while another process reads the data from it. The project demonstrates the creation of pipes and the use of Linux system calls such as read(), write(), and close() to transfer information between processes.

FIFOs, also known as Named Pipes, provide communication between independent processes. Unlike ordinary pipes, FIFOs have a name in the file system, allowing unrelated processes to access the same communication channel. The project demonstrates how FIFOs can be created with mkfifo() and used to send and receive data between separate processes, with proper opening, reading, writing and closing of the channel.

Overall, this project provides practical knowledge of process communication, synchronization, data sharing, and Linux system calls. It helps in understanding how the Linux operating system manages communication between processes and how Pipes and FIFOs can be used to build efficient and reliable multi-process applications. The programs are written in C, compiled with GCC and executed on Ubuntu Linux, and display reliable data transfer between processes in the terminal.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–9
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	8
		1.4 Objectives of the Project	9
		1.5 Importance of the Application	9
		1.6 Target Users / Audience	9
2	OS Concepts and Background Study		10–12
		2.1 Operating System Concepts Related to the Project	10
		2.2 Existing Methods / Algorithms	11
		2.3 Proposed Approach	11
		2.4 Advantages of the Proposed Approach	12
3	System Requirements		13–14
		3.1 Hardware Requirements	13
		3.2 Software Requirements	13
		3.3 Operating System Environment – Ubuntu / WSL	13
		3.4 Development Tools and Compilers	14
4	Technology and Tools Used		15–16
		4.1 C / C++ Programming	15
		4.2 GCC Compiler	15
		4.3 Linux System Calls	15
		4.4 POSIX Threads / Synchronization	15
		4.5 Linux Commands and Utilities	16
		4.6 Other Tools / Libraries Used	16
5	System Architecture and Design		17–21
		5.1 Overall System Architecture	17
		5.2 System Workflow	18

		5.3 Flowchart / Data Flow	**19**
		5.4 Module Description	**20**
		5.5 Input and Output Design	**20**
6	**Implementation**		**22–26**
		6.1 Module-wise Implementation	**22**
		6.2 Algorithm / Pseudocode	**22**
		6.3 Core C / C++ Implementation	**23**
		6.4 Linux System Calls / APIs Used	**24**
		6.5 Process / Thread Implementation	**25**
		6.6 IPC / Synchronization Implementation	**25**
		6.7 Memory Management Implementation	**25**
		6.8 Optional User Interface Implementation	**26**
7	**Execution and Demonstration**		**27–29**
		7.1 Compilation in Ubuntu	**27**
		7.2 Execution Commands	**27**
		7.3 Sample Inputs and Outputs	**27**
		7.4 Screenshots of Execution	**28**
		7.5 Demonstration of OS Concepts	**29**
8	**Conclusion**		**30–31**
		8.1 Summary of the Project	**30**
		8.2 Objectives Achieved	**30**
		8.3 Results Achieved	**30**
		8.4 Skills and Learning Outcomes	**30**
9	**References**		**32**
		9.1 Books	**32**
		9.2 Research Papers	**32**
		9.3 Linux / GNU Documentation	**32**
		9.4 Online Resources	**32**

CHAPTER - 1
INTRODUCTION

1.1 Background of the Project

A modern operating system runs many programs at the same time. Each running program is called a **process**, and the operating system gives every process its own private virtual address space so that one process cannot accidentally read or overwrite the memory of another. This isolation is essential for security and stability, but it creates an obvious difficulty: real applications are rarely a single process. A web server, a shell pipeline such as `ls | grep txt`, a database and its clients, or a producer–consumer application all need their processes to exchange data and coordinate their work.

The mechanisms that the operating system provides for this purpose are collectively called **Inter-Process Communication (IPC)**. Linux supports many IPC mechanisms, including pipes, FIFOs (named pipes), message queues, shared memory, semaphores, signals and sockets. Among them, **pipes** and **FIFOs** are the oldest, simplest and most widely used. The pipe was introduced in the early UNIX system and is still the basis of the shell's `|` operator; the FIFO extends the same idea by giving the channel a name in the file system.

This project, **"Linux Inter-Process Communication Using Pipes and FIFOs"**, studies and implements these two mechanisms in C using Linux system calls such as `pipe()`, `fork()`, `mkfifo()`, `open()`, `read()`, `write()` and `close()`. The programs are developed and executed on Ubuntu Linux and demonstrate how data actually moves between processes through the kernel.

1.2 Problem Statement

In Linux, processes normally have separate memory spaces, so one process cannot directly share variables or buffers with another. Therefore an efficient and safe IPC mechanism is required. The main problem addressed in this project is to understand and demonstrate how processes can transfer information safely, coordinate their execution and communicate efficiently without directly accessing each other's memory.

The project develops a working system that demonstrates communication between **related processes** (a parent and its child) using **pipes**, and between **independent processes** (two unrelated programs started separately) using **FIFOs (named pipes)**. It also highlights the differences between the two methods and the situations in which each one is appropriate.

1.3 Scope of the Project

- Implementation of an anonymous pipe between a parent and a child process created with

`fork()`.

- Implementation of a named pipe (FIFO) between two independent processes – a writer program and a reader program.
- Demonstration of blocking behaviour, end-of-file detection and proper closing of unused descriptors.
- Execution and testing in the Linux terminal on Ubuntu using the GCC compiler.
- The project is limited to one-way (unidirectional) communication for each channel. Other IPC mechanisms (shared memory, message queues, sockets) are discussed only for comparison and are not implemented.

1.4 Objectives of the Project

- To understand the concept of Inter-Process Communication (IPC) in the Linux operating system.
- To implement Pipes for communication and data exchange between related processes.
- To implement FIFOs (Named Pipes) for communication between independent processes.
- To demonstrate process synchronization and data sharing using Linux IPC system calls such as `read()`, `write()` and `close()`.

1.5 Importance of the Application

IPC is a core topic of Operating Systems and Systems Programming. Almost every non-trivial Linux application relies on it. Learning how pipes and FIFOs work gives a practical understanding of file descriptors, process creation, kernel buffering, blocking I/O and synchronization – concepts that are used in shells, daemons, logging systems, client–server programs and multi-process applications. The project also shows how the UNIX philosophy of "everything is a file" allows a communication channel to be accessed with the same `read()` and `write()` calls used for ordinary files.

1.6 Target Users / Audience

- **Students** of Operating Systems and Systems Programming who want a working reference for IPC.
- **Faculty and evaluators** who need a clear demonstration of pipes and FIFOs.
- **Beginner Linux/C programmers** learning multi-process programming.
- **Developers** who need a lightweight IPC mechanism for simple producer–consumer applications.

CHAPTER - 2
OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and Process Management

A process is a program in execution. In Linux a new process is created with `fork()`, which duplicates the calling process. The original is the **parent** and the copy is the **child**; `fork()` returns the child's process ID to the parent and 0 to the child, which lets the two branches of the program behave differently. The parent can wait for the child with `wait()`, which prevents zombie processes.

Separate Address Spaces

After `fork()`, parent and child have independent copies of memory. A variable changed in one process is not changed in the other. For this reason ordinary variables cannot be used to exchange data, and the kernel must provide a communication channel.

File Descriptors

A file descriptor is a small non-negative integer that a process uses to refer to an open file, device, pipe or socket. Descriptors 0, 1 and 2 are standard input, output and error. Both pipes and FIFOs are accessed through file descriptors, and descriptors are **inherited** by a child created with `fork()`. This inheritance is what allows a parent and child to share a pipe.

Kernel Buffering and Blocking I/O

A pipe or FIFO is implemented as a buffer inside the kernel (64 KB by default on Linux). A `read()` on an empty pipe **blocks** until data arrives; a `write()` on a full pipe blocks until space is available. This blocking behaviour is the natural form of synchronization between the writer and the reader. When all write ends are closed, `read()` returns 0, which signals end-of-file.

Process Synchronization

Synchronization ensures that cooperating processes execute in a correct order and do not conflict. In this project it is achieved by (i) the blocking semantics of `read()`/`open()`, (ii) the EOF signal produced by `close()`, and (iii) `wait()` in the parent. No additional locks are needed because the kernel serialises access to the pipe buffer.

The table below summarises the OS concepts used in the project.

OS Concept / Linux API	Purpose in the Project
Process Management	Creation and management of processes using `fork()`; waiting with `wait()`.
Inter-Process Communication (IPC)	Communication between processes using Pipes and FIFOs.

OS Concept / Linux API	Purpose in the Project
Pipes	Communication between related processes using `pipe()`.
FIFOs (Named Pipes)	Communication between independent processes using `mkfifo()`.
File Descriptors	Used to identify and access communication channels.
Process Synchronization	Ensures processes communicate in the correct sequence and avoid conflicts.

2.2 Existing Methods / Algorithms

Linux offers several IPC methods. Each has its own strengths and costs, as compared below.

IPC Method	How it works	Limitation / Remark
Shared Memory	Processes map the same memory region and read/write it directly.	Fastest, but needs explicit synchronization (semaphores/mutexes).
Message Queues	Kernel-managed queue of typed messages.	Message size limits; more complex API.
Semaphores	Counters used to coordinate access to resources.	Provide synchronization only, not data transfer.
Signals	Asynchronous notification sent to a process.	Carry almost no data.
Sockets	Endpoint for communication, local or over a network.	Heavier setup; suited to client–server designs.
Regular Files	One process writes a file, another reads it.	Slow, needs polling, risk of race conditions.
Pipes	Unidirectional kernel buffer between related processes.	Only between processes with a common ancestor.
FIFOs	Pipe with a name in the file system.	Unidirectional; blocks until both ends are open.

2.3 Proposed Approach

The proposed approach implements the two simplest kernel-supported IPC channels and demonstrates them with small, clear C programs.

- **Step 1 – Pipe between related processes.** A pipe is created with `pipe(fd)`. The program calls `fork()`. The parent closes the read end and writes a message to `fd[1]`; the child closes the write end and reads the message from `fd[0]`. Unused ends are closed so that EOF is delivered correctly.
- **Step 2 – FIFO between independent processes.** A FIFO is created with `mkfifo()`. A separate writer program opens it with `O_WRONLY` and a separate reader program opens it with `O_RDONLY`. The two programs are started independently from different terminals and exchange messages through the named channel.

- **Step 3 – Synchronization and cleanup.** Blocking `open()`/`read()` provide ordering; the reader removes the FIFO with `unlink()` after use.
- **Step 4 – Testing and documentation.** The programs are compiled with GCC, tested with different inputs, and the results are captured as outputs and screenshots.

2.4 Advantages of the Proposed Approach

- **Simplicity** – only a handful of system calls are needed, which makes the code easy to read and verify.
- **Kernel-managed synchronization** – blocking semantics remove the need for explicit locks.
- **Safety** – processes never touch each other's memory; all data passes through the kernel.
- **Uniform interface** – the same `read()`/`write()`/`close()` calls work for pipes, FIFOs and files.
- **Flexibility** – pipes serve parent–child communication; FIFOs serve unrelated processes.
- **No extra libraries** – the programs use only standard POSIX/Linux headers.

CHAPTER - 3
SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The programs are lightweight and run on any modern computer. The following configuration is sufficient.

Component	Minimum	Recommended
Processor	Dual-core, 1.5 GHz	Quad-core, 2.0 GHz or higher
RAM	2 GB	4 GB or more
Storage	10 GB free disk space	20 GB or more
Display	Standard monitor with terminal access	Full HD monitor
Input devices	Keyboard and mouse	Keyboard and mouse

3.2 Software Requirements

Software	Details
Operating System	Linux (Ubuntu 20.04 LTS or later) or Windows with WSL2 running Ubuntu
Programming Language	C (the programs are also compatible with C++)
Compiler	GCC (GNU Compiler Collection)
Editor / IDE	nano, vim, gedit or Visual Studio Code
Shell / Terminal	Bash
Version Control	Git and GitHub

3.3 Operating System Environment – Ubuntu / WSL

The project is developed on **Ubuntu Linux**, a Debian-based distribution that ships with the Linux kernel, the GNU C library (glibc) and the Bash shell. Pipes, FIFOs and `fork()` are native kernel features, so no additional configuration is required.

Team members who use Windows can run the same code through the **Windows Subsystem for Linux (WSL2)**, which provides a real Linux kernel. After installing Ubuntu from the Microsoft Store, the build tools can be installed with:

```
sudo apt update
sudo apt install build-essential
```

FIFOs must be created on a Linux file system (for example `/tmp` or the home directory inside WSL). They cannot be created on mounted Windows drives such as `/mnt/c`.

3.4 Development Tools and Compilers

- **GCC** – compiles the C source files into executables (`gcc file.c -o file`).
- **GNU Make** (optional) – automates compilation of the three programs.
- **GDB** (optional) – debugging of process behaviour.
- **strace** (optional) – traces system calls to verify `pipe()`, `read()` and `write()` activity.
- **Git / GitHub** – version control and sharing of the source code among the team.

CHAPTER - 4
TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

C is the native language of UNIX and Linux, and the system-call interface of the kernel is exposed to programmers through C functions. C gives direct access to file descriptors, process creation and byte buffers without any runtime overhead, which makes it the ideal language for demonstrating IPC. The programs in this project use the headers `<unistd.h>`, `<sys/wait.h>`, `<sys/stat.h>`, `<fcntl.h>`, `<stdio.h>`, `<stdlib.h>` and `<string.h>`. C++ can be used with identical system calls.

4.2 GCC Compiler

The GNU Compiler Collection (GCC) is used to compile and link the programs. Warnings are enabled with `-Wall -Wextra` so that common mistakes (unused variables, ignored return values, wrong types) are caught during compilation.

```
gcc -Wall -Wextra pipe_demo.c   -o pipe_demo
gcc -Wall -Wextra fifo_writer.c -o fifo_writer
gcc -Wall -Wextra fifo_reader.c -o fifo_reader
```

4.3 Linux System Calls

A system call is the controlled entry point through which a user program requests a service from the kernel. The calls used in this project are:

System Call	Header	Purpose
`pipe(int fd[2])`	`<unistd.h>`	Creates an anonymous pipe; `fd[0]` is the read end, `fd[1]` the write end.
`fork()`	`<unistd.h>`	Creates a child process that inherits the descriptors.
`mkfifo(path, mode)`	`<sys/stat.h>`	Creates a named pipe (FIFO) in the file system.
`open(path, flags)`	`<fcntl.h>`	Opens a FIFO for reading (`O_RDONLY`) or writing (`O_WRONLY`).
`read(fd, buf, n)`	`<unistd.h>`	Reads up to n bytes; blocks if empty; returns 0 at EOF.
`write(fd, buf, n)`	`<unistd.h>`	Writes n bytes into the pipe or FIFO.
`close(fd)`	`<unistd.h>`	Closes a descriptor; closing all write ends produces EOF.
`wait(NULL)`	`<sys/wait.h>`	Parent waits for the child to terminate.
`unlink(path)`	`<unistd.h>`	Removes the FIFO name from the file system.

4.4 POSIX Threads / Synchronization

POSIX threads (`pthread`) provide mutexes and condition variables for synchronizing threads

that share memory. The present project communicates between **processes** that do not share memory, so threads are not required. Synchronization is obtained from the semantics of the IPC objects themselves:

- `read()` blocks until the writer has produced data (consumer waits for producer).
- `open()` on a FIFO blocks until the opposite end is opened (rendezvous).
- `close()` on the last write end delivers EOF to the reader.
- `wait()` makes the parent wait until the child finishes.

If the project were extended to multiple writer threads, `pthread_mutex_t` could be added to serialise their writes; writes of at most `PIPE_BUF` (4096) bytes are, however, guaranteed to be atomic by POSIX.

4.5 Linux Commands and Utilities

Command	Use in the project
`gcc`	Compile the C programs.
`./program`	Run the compiled executable.
`mkfifo name`	Create a FIFO from the shell (alternative to the `mkfifo()` call).
`ls -l /tmp/myfifo`	Verify that the FIFO exists (file type `p`).
`ps`, `top`	View running processes and their IDs.
`strace -f ./pipe_demo`	Trace the system calls made by parent and child.
`cat`, `echo`	Send or display data through a FIFO from the shell.
`rm`	Remove an unused FIFO.

4.6 Other Tools / Libraries Used

- **Git and GitHub** – the project repository is hosted at https://github.com/bdeepakg-boop/KLH-AI-DS-2026-TEAM5-FIFO-IPC.
- **Terminal emulator** – used to start multiple processes side by side (one terminal for the writer, one for the reader).
- **Text editor / VS Code** – source-code editing.
- **Standard C library (glibc)** – `printf`, `fgets`, `strlen`, `strncmp`, `perror`.

CHAPTER - 5
SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system consists of two independent IPC demonstrations. In the **pipe module** a parent and child process communicate through an anonymous kernel buffer. In the **FIFO module** two separate programs communicate through a named kernel buffer that is visible in the file system. In both cases the user-space processes never access each other's memory – all data passes through the kernel.

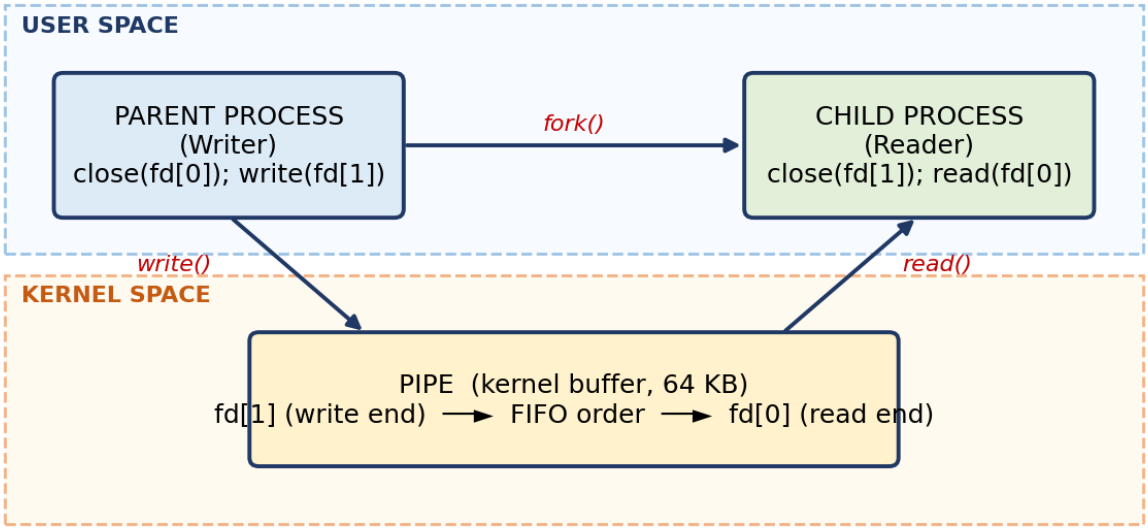


Figure 5.1: Architecture of pipe-based communication (parent → child)

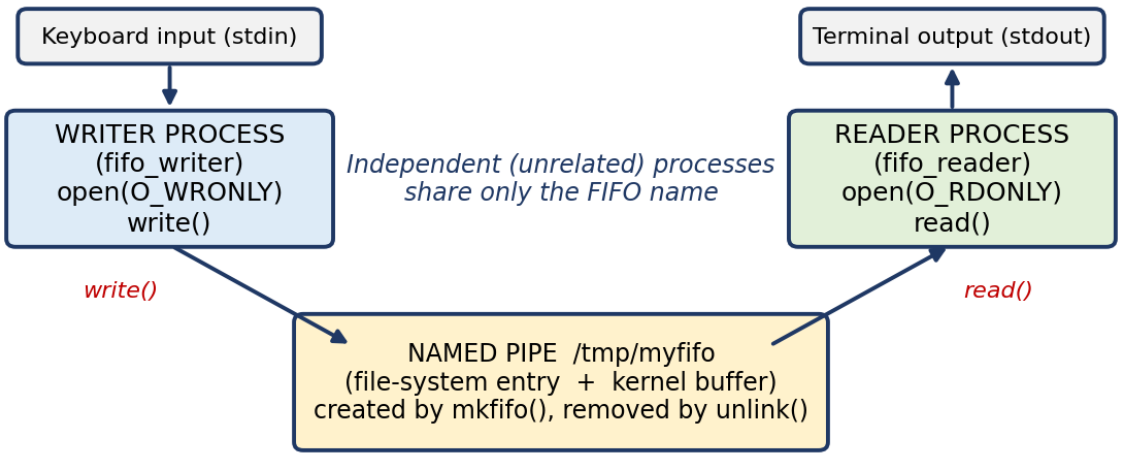


Figure 5.2: Architecture of FIFO-based communication (independent processes)

Feature	Pipe	FIFO (Named Pipe)
Created by	`pipe()`	`mkfifo()`
Name in file system	No (anonymous)	Yes (e.g. `/tmp/myfifo`)

Feature	Pipe	FIFO (Named Pipe)
Processes allowed	Related (common ancestor, via `fork()`)	Any processes, related or unrelated
Lifetime	Until all descriptors are closed	Until removed with `unlink()` / `rm`
Direction	Unidirectional	Unidirectional
Access method	Inherited descriptors	`open()` by pathname

5.2 System Workflow

Pipe Workflow

- The program creates a pipe with `pipe(fd)`.
- `fork()` creates the child, which inherits both descriptors.
- Parent closes `fd[0]`, writes the message to `fd[1]`, closes `fd[1]` and calls `wait()`.
- Child closes `fd[1]`, reads the message from `fd[0]`, displays it and exits.

FIFO Workflow

- The reader (or writer) creates the FIFO with `mkfifo("/tmp/myfifo")`.
- The reader opens it with `O_RDONLY` and blocks until a writer appears.
- The writer opens it with `O_WRONLY`; both `open()` calls now return.
- The writer sends each line typed by the user; the reader prints each line as it arrives.
- When the line `exit` is sent, both programs close the FIFO and the reader removes it with `unlink()`.

5.3 Flowchart / Data Flow

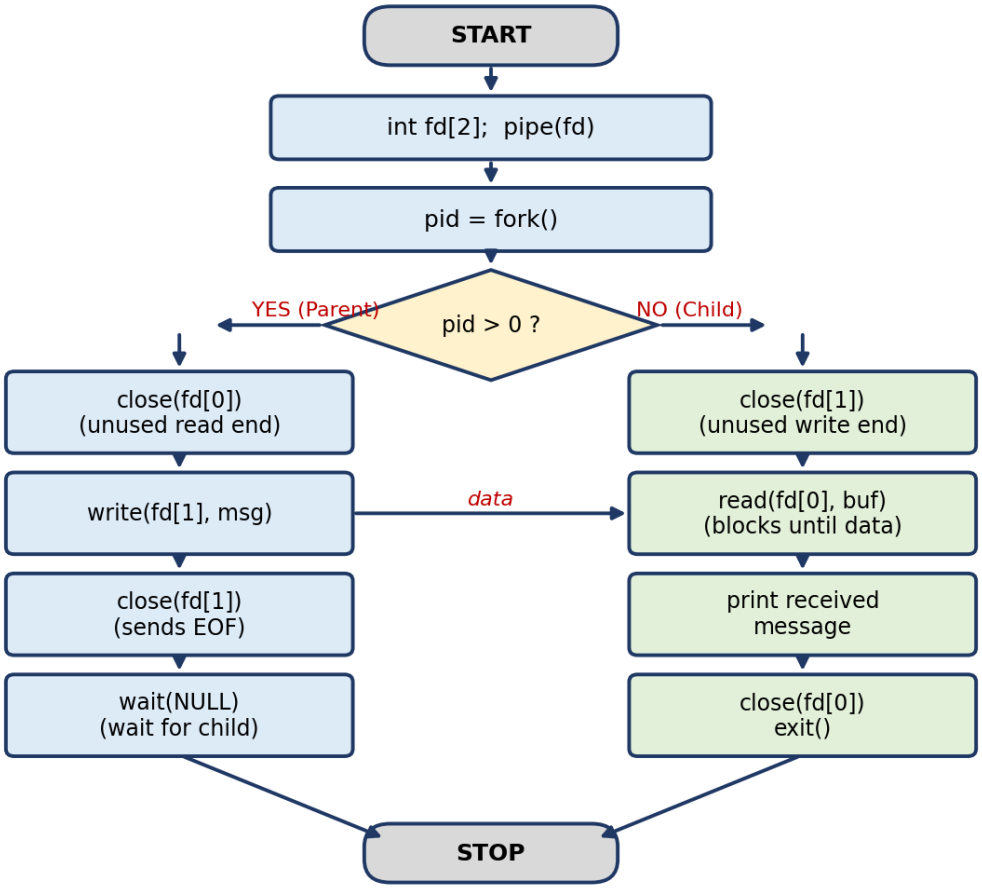


Figure 5.3: Flowchart of the pipe program (pipe_demo.c)

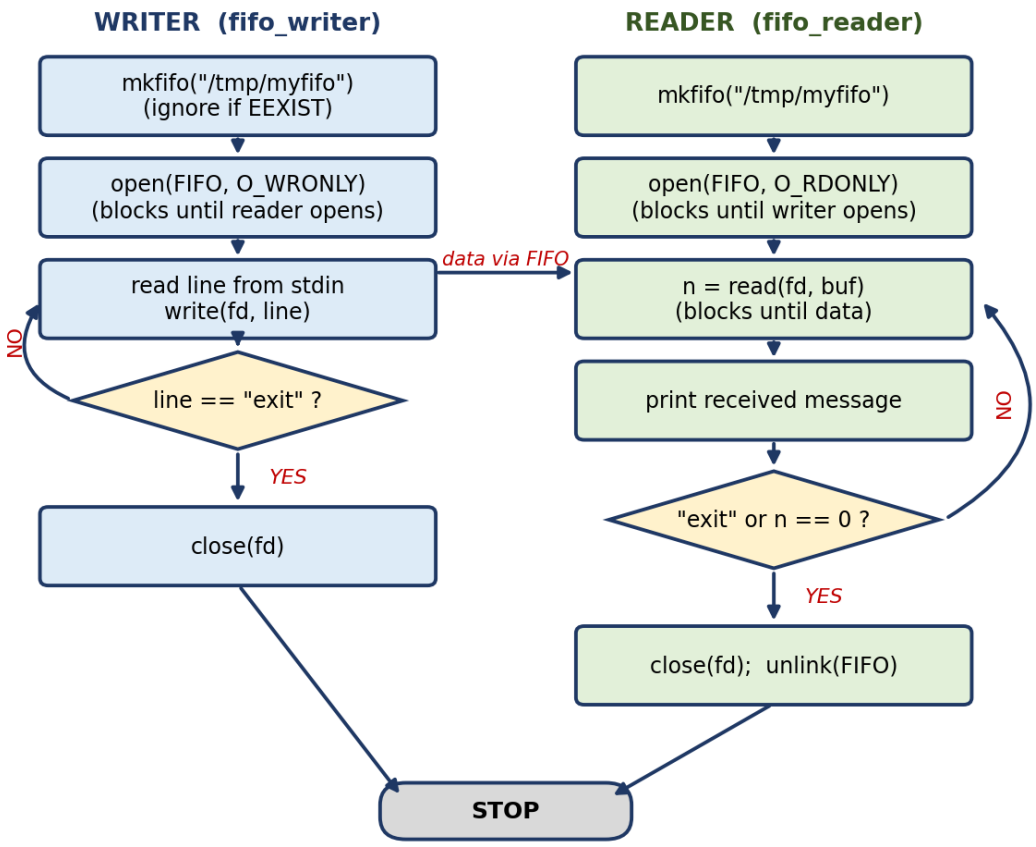


Figure 5.4: Flowchart of the FIFO writer and reader programs

Data flow: keyboard / string constant → `write()` → kernel pipe buffer (FIFO order, first-in first-out) → `read()` → user buffer in the reader → `printf()` → terminal.

5.4 Module Description

Module	Source file	Description	Team member
Pipe Communication Module	`pipe_demo.c`	Creates a pipe, forks a child; parent writes, child reads. Demonstrates descriptor inheritance, closing of unused ends and EOF.	B. Deepak
FIFO Communication Module	`fifo_writer.c` `fifo_reader.c`	Creates a named pipe and lets two independent processes exchange messages; handles termination keyword and cleanup.	E. Pranay Reddy
Testing & Documentation Module	All programs	Testing, debugging, preparation of documentation, screenshots and presentation.	A. Lakshmi Sai Teja

5.5 Input and Output Design

Program	Input	Output
`pipe_demo`	Fixed message string inside the program	Parent and child print their process IDs and the received message with its byte count.

Program	Input	Output
`fifo_writer`	Lines typed by the user at the keyboard; `exit` ends the session	Status messages on the writer terminal.
`fifo_reader`	Data arriving through `/tmp/myfifo`	Each received line is printed on the reader terminal; a closing message is shown after cleanup.

CHAPTER - 6
IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is divided into three small C programs, each with a single responsibility. All three check the return value of the system calls they use and report errors with `perror()`.

- **Module 1 – `pipe_demo.c`:** pipe + `fork()` between parent and child (Section 6.3.1).
- **Module 2 – `fifo_writer.c`:** creates the FIFO and sends user input (Section 6.3.2).
- **Module 3 – `fifo_reader.c`:** receives the data, prints it and removes the FIFO (Section 6.3.3).

6.2 Algorithm / Pseudocode

Algorithm 1: Pipe communication

```
1.  START
2.  Declare fd[2] and buffer
3.  Create pipe: if pipe(fd) fails -> print error, exit
4.  pid = fork(); if pid < 0 -> print error, exit
5.  IF pid > 0 (PARENT)
6.      close(fd[0])                 // read end not needed
7.      write message into fd[1]
8.      close(fd[1])                 // delivers EOF
9.      wait for child to finish
10. ELSE (CHILD)
11.     close(fd[1])                 // write end not needed
12.     n = read(fd[0], buffer)      // blocks until data
13.     display buffer
14.     close(fd[0]); exit
15. STOP
```

Algorithm 2: FIFO writer

```
1.  START
2.  mkfifo("/tmp/myfifo", 0666)        // ignore error if it already exists
3.  fd = open(FIFO, O_WRONLY)          // blocks until a reader opens
4.  WHILE a line is available on stdin
5.      write(fd, line)
6.      IF line begins with "exit" THEN break
7.  close(fd)
8.  STOP
```

Algorithm 3: FIFO reader

```
1.  START
2.  mkfifo("/tmp/myfifo", 0666)        // create if not present
3.  fd = open(FIFO, O_RDONLY)          // blocks until a writer opens
4.  WHILE (n = read(fd, buffer)) > 0
5.      terminate buffer with '\0' and display it
6.      IF buffer begins with "exit" THEN break
7.  close(fd)
8.  unlink(FIFO)                       // remove FIFO from file system
9.  STOP
```

6.3 Core C / C++ Implementation

6.3.1 pipe_demo.c – Pipe between parent and child

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define BUF_SIZE 128

int main(void) {
    int fd[2];                      /* fd[0] = read end, fd[1] = write end */
    char buffer[BUF_SIZE];
    pid_t pid;

    if (pipe(fd) == -1) {           /* create the pipe */
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid = fork();                   /* create the child process */
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0) {                  /* ---- PARENT: writer ---- */
        const char *msg = "Hello from Parent process via PIPE";
        close(fd[0]);               /* close unused read end */
        printf("[Parent PID %d] Writing message to pipe...\n", getpid());
        fflush(stdout);
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);               /* closing signals EOF to the reader */
        wait(NULL);                 /* wait for the child to finish */
        printf("[Parent PID %d] Child finished. Parent exiting.\n", getpid());
    } else {                        /* ---- CHILD: reader ---- */
        close(fd[1]);               /* close unused write end */
        ssize_t n = read(fd[0], buffer, BUF_SIZE);
        if (n > 0)
            printf("[Child  PID %d] Received %zd bytes: \"%s\"\n", getpid(), n, buffer);
        close(fd[0]);
        exit(EXIT_SUCCESS);
    }
    return 0;
}
```

6.3.2 fifo_writer.c – FIFO writer process

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <errno.h>

#define FIFO_PATH "/tmp/myfifo"

int main(void) {
    char msg[128];

    if (mkfifo(FIFO_PATH, 0666) == -1 && errno != EEXIST) {   /* create the named pipe */
        perror("mkfifo");
        exit(EXIT_FAILURE);
    }
```

```
    printf("[Writer] Waiting for a reader to open the FIFO...\n");
    int fd = open(FIFO_PATH, O_WRONLY);       /* blocks until reader opens */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }

    printf("[Writer] Connected. Type messages (type 'exit' to quit):\n");
    while (fgets(msg, sizeof(msg), stdin) != NULL) {
        write(fd, msg, strlen(msg));
        if (strncmp(msg, "exit", 4) == 0)
            break;
    }
    close(fd);
    printf("[Writer] Closed FIFO.\n");
    return 0;
}
```

6.3.3 fifo_reader.c – FIFO reader process

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_PATH "/tmp/myfifo"

int main(void) {
    char buffer[128];

    mkfifo(FIFO_PATH, 0666);                  /* create if not present */
    printf("[Reader] Waiting for a writer to open the FIFO...\n");
    int fd = open(FIFO_PATH, O_RDONLY);       /* blocks until writer opens */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }

    printf("[Reader] Connected. Waiting for messages...\n");
    ssize_t n;
    while ((n = read(fd, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[n] = '\0';
        printf("[Reader] Received: %s", buffer);
        if (strncmp(buffer, "exit", 4) == 0)
            break;
    }
    close(fd);
    unlink(FIFO_PATH);                        /* remove the FIFO from the file system */
    printf("[Reader] FIFO closed and removed.\n");
    return 0;
}
```

6.4 Linux System Calls / APIs Used

System Call	Used in	Role
`pipe()`	`pipe_demo.c`	Creates the pipe and fills `fd[0]`, `fd[1]`.
`fork()`	`pipe_demo.c`	Creates the child process.
`wait()`	`pipe_demo.c`	Parent waits for child termination.
`mkfifo()`	Writer, Reader	Creates `/tmp/myfifo` with permissions 0666 (modified by umask).
`open()`	Writer, Reader	Opens the FIFO; blocks until the other end is open.
`write()`	All	Transfers bytes into the kernel buffer.
`read()`	All	Retrieves bytes from the kernel buffer.
`close()`	All	Releases descriptors; last writer close → EOF.

System Call	Used in	Role
`unlink()`	Reader	Deletes the FIFO entry.
`getpid()`	`pipe_demo.c`	Displays process IDs in the output.
`perror()`	All	Prints a readable error message when a call fails.

6.5 Process / Thread Implementation

Processes. In `pipe_demo.c` a single `fork()` produces two processes that run the same code but take different branches according to the value returned by `fork()`. The parent acts as the producer and the child as the consumer. The parent calls `wait(NULL)` so that it does not terminate before its child and no zombie process is left behind.

In the FIFO module the two processes are **not** related by `fork()`; they are two separate programs started from two terminals. Their only link is the pathname `/tmp/myfifo`.

Threads. The project is process-based and does not create threads. The implementation can be extended with `pthread_create()` if multiple concurrent writers are required.

6.6 IPC / Synchronization Implementation

- **Closing unused ends.** Each process closes the pipe end it does not use. This is essential: if the parent kept `fd[1]` open, the child's `read()` would never see EOF and would block forever.
- **Blocking `open()`.** Opening a FIFO for reading blocks until a writer opens it, and vice versa, so the two programs automatically synchronise at start-up regardless of which is launched first.
- **Blocking `read()`.** The reader sleeps inside the kernel until data is written; no busy waiting or polling is used.
- **EOF detection.** `read()` returns 0 when every write end has been closed, which ends the loop cleanly.
- **Termination protocol.** The keyword `exit` is sent through the FIFO so that both programs finish in a controlled way.
- **Atomic writes.** Messages shorter than `PIPE_BUF` (4096 bytes) are written atomically, so lines are never interleaved or split.
- **Broken pipe.** Writing to a pipe with no reader raises `SIGPIPE` (error `EPIPE`); the programs avoid this by always keeping the reader open until the writer is finished.

6.7 Memory Management Implementation

The programs use only **stack memory** for their buffers (`char buffer[128]`), so no dynamic allocation (`malloc`/`free`) is needed and memory leaks are not possible. Message size is limited by `BUF_SIZE`, and the reader leaves one byte free for the terminating `'\0'` to prevent buffer overflow. Data in transit is held in the kernel's pipe buffer (64 KB by default); the kernel allocates and frees it automatically when the descriptors are closed. After `fork()` the child receives a copy-on-write copy of the parent's address space, so no memory is copied until a process modifies it.

6.8 Optional User Interface Implementation

The project uses a **command-line interface**. In the FIFO demonstration the user types messages in the writer terminal and sees them appear in the reader terminal. Status lines prefixed with `[Writer]`, `[Reader]`, `[Parent PID …]` and `[Child PID …]` make it easy to follow which process is acting. No graphical interface is required for demonstrating the OS concepts.

CHAPTER - 7
EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open a terminal in the project folder and compile each program with GCC:

```
$ gcc -Wall -Wextra pipe_demo.c   -o pipe_demo
$ gcc -Wall -Wextra fifo_writer.c -o fifo_writer
$ gcc -Wall -Wextra fifo_reader.c -o fifo_reader
$ ls
fifo_reader  fifo_reader.c  fifo_writer  fifo_writer.c  pipe_demo  pipe_demo.c
```

All three programs compile without errors or warnings.

7.2 Execution Commands

Pipe program

```
$ ./pipe_demo
```

FIFO programs (two terminals)

```
# Terminal 1 – start the reader first (or second; order does not matter)
$ ./fifo_reader

# Terminal 2 – start the writer and type messages
$ ./fifo_writer

# Optional – verify the FIFO from a third terminal
$ ls -l /tmp/myfifo
prw-r--r-- 1 user user 0 ... /tmp/myfifo
```

The first character `p` in the `ls -l` output shows that the file is a named pipe.

7.3 Sample Inputs and Outputs

Output of pipe_demo

```
$ ./pipe_demo
[Parent PID 220] Writing message to pipe...
[Child  PID 222] Received 35 bytes: "Hello from Parent process via PIPE"
[Parent PID 220] Child finished. Parent exiting.
```

(Process IDs differ on every run.)

Output of the FIFO programs

Input typed in the writer terminal:

```
Hello Reader
IPC using FIFO works
exit
```

Writer terminal output:

```
[Writer] Waiting for a reader to open the FIFO...
[Writer] Connected. Type messages (type 'exit' to quit):
[Writer] Closed FIFO.
```

Reader terminal output:

```
[Reader] Waiting for a writer to open the FIFO...
[Reader] Connected. Waiting for messages...
[Reader] Received: Hello Reader
[Reader] Received: IPC using FIFO works
[Reader] Received: exit
[Reader] FIFO closed and removed.
```

7.4 Screenshots of Execution

Replace the placeholders below with screenshots taken on the Ubuntu terminal.

[Paste screenshot here]
Screenshot 1: Compilation of the three programs

Figure 7.1: Compilation using GCC

[Paste screenshot here]
Screenshot 2: Output of pipe_demo

Figure 7.2: Parent–child communication using a pipe

[Paste screenshot here]
Screenshot 3: Reader and writer terminals side by side

Figure 7.3: Communication between independent processes using a FIFO

[Paste screenshot here]

Screenshot 4: ls -l /tmp/myfifo showing the file type p

Figure 7.4: The FIFO as seen in the file system

7.5 Demonstration of OS Concepts

OS Concept	How the demonstration shows it
Process creation	`fork()` produces two processes with different PIDs, visible in the output.
Separate address spaces	Data reaches the child only through the pipe, not through shared variables.
File descriptors	`fd[0]` and `fd[1]` are inherited by the child; the FIFO is accessed through the descriptor returned by `open()`.
Blocking I/O	The reader waits at `open()`/`read()` until the writer supplies data.
Synchronization	Messages appear in the reader in the same order they were written; `wait()` orders parent and child termination.
EOF / resource release	`close()` of the write end terminates the reader loop; `unlink()` removes the FIFO.
Related vs independent processes	Pipe needs `fork()`; the FIFO works between unrelated programs started separately.

CHAPTER - 8
CONCLUSION

8.1 Summary of the Project

This project demonstrated Inter-Process Communication in Linux using Pipes and FIFOs. An anonymous pipe was used to pass data from a parent process to its child, and a named pipe (FIFO) was used to exchange messages between two completely independent programs. The implementation made use of the `pipe()`, `fork()`, `mkfifo()`, `open()`, `read()`, `write()`, `close()`, `wait()` and `unlink()` system calls, and showed how the kernel provides buffering and synchronization without requiring processes to share memory.

Limitations and future scope. Each channel is unidirectional; two-way communication would need a pair of pipes or FIFOs. The project could be extended with multiple writers, a chat application, a producer–consumer server using non-blocking I/O (`O_NONBLOCK`, `select()`/`poll()`), and a comparison of throughput against shared memory and message queues.

8.2 Objectives Achieved

Objective	Status	Where achieved
Understand the concept of IPC in Linux	Achieved	Chapter 2
Implement Pipes for related processes	Achieved	`pipe_demo.c` – Sections 6.3.1, 7.3
Implement FIFOs for independent processes	Achieved	`fifo_writer.c`, `fifo_reader.c` – Sections 6.3.2, 6.3.3, 7.3
Demonstrate synchronization and data sharing with `read()`, `write()`, `close()`	Achieved	Sections 6.6 and 7.5

8.3 Results Achieved

- The pipe program transferred a 35-byte message from parent to child correctly on every run.
- The FIFO reader and writer exchanged multiple lines in the correct order and terminated cleanly with the `exit` keyword.
- The FIFO was removed from the file system after use, leaving no residue.
- All programs compiled without warnings using `-Wall -Wextra` and ran successfully in the Ubuntu terminal.

8.4 Skills and Learning Outcomes

- Practical understanding of processes, `fork()` and `wait()`.

- Working knowledge of file descriptors and the “everything is a file” principle.
- Understanding of blocking I/O, EOF and kernel buffering.
- Ability to choose between pipes and FIFOs for a given problem.
- Experience with C system programming, GCC, Linux command-line tools and Git/GitHub.
- Teamwork, debugging, documentation and presentation skills.

CHAPTER - 9
REFERENCES

9.1 Books

- A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
- M. Kerrisk, The Linux Programming Interface, No Starch Press, 2010.
- W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.
- R. Love, Linux System Programming, 2nd ed., O'Reilly Media, 2013.
- A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.

9.2 Research Papers

- D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," Communications of the ACM, vol. 17, no. 7, pp. 365–375, 1974.
- D. M. Ritchie, "The Evolution of the Unix Time-sharing System," AT&T Bell Laboratories Technical Journal, vol. 63, no. 8, 1984.

9.3 Linux / GNU Documentation

- Linux man-pages: pipe(2), fork(2), read(2), write(2), close(2), open(2), wait(2), unlink(2).
- Linux man-pages: mkfifo(3), pipe(7), fifo(7).
- The GNU C Library Reference Manual – Pipes and FIFO Special Files.
- GCC, the GNU Compiler Collection – Online Documentation.

9.4 Online Resources

- Linux man-pages online: https://man7.org/linux/man-pages/
- GNU C Library Manual: https://www.gnu.org/software/libc/manual/
- GCC Documentation: https://gcc.gnu.org/onlinedocs/
- Project repository: https://github.com/bdeepakg-boop/KLH-AI-DS-2026-TEAM5-FIFO-IPC